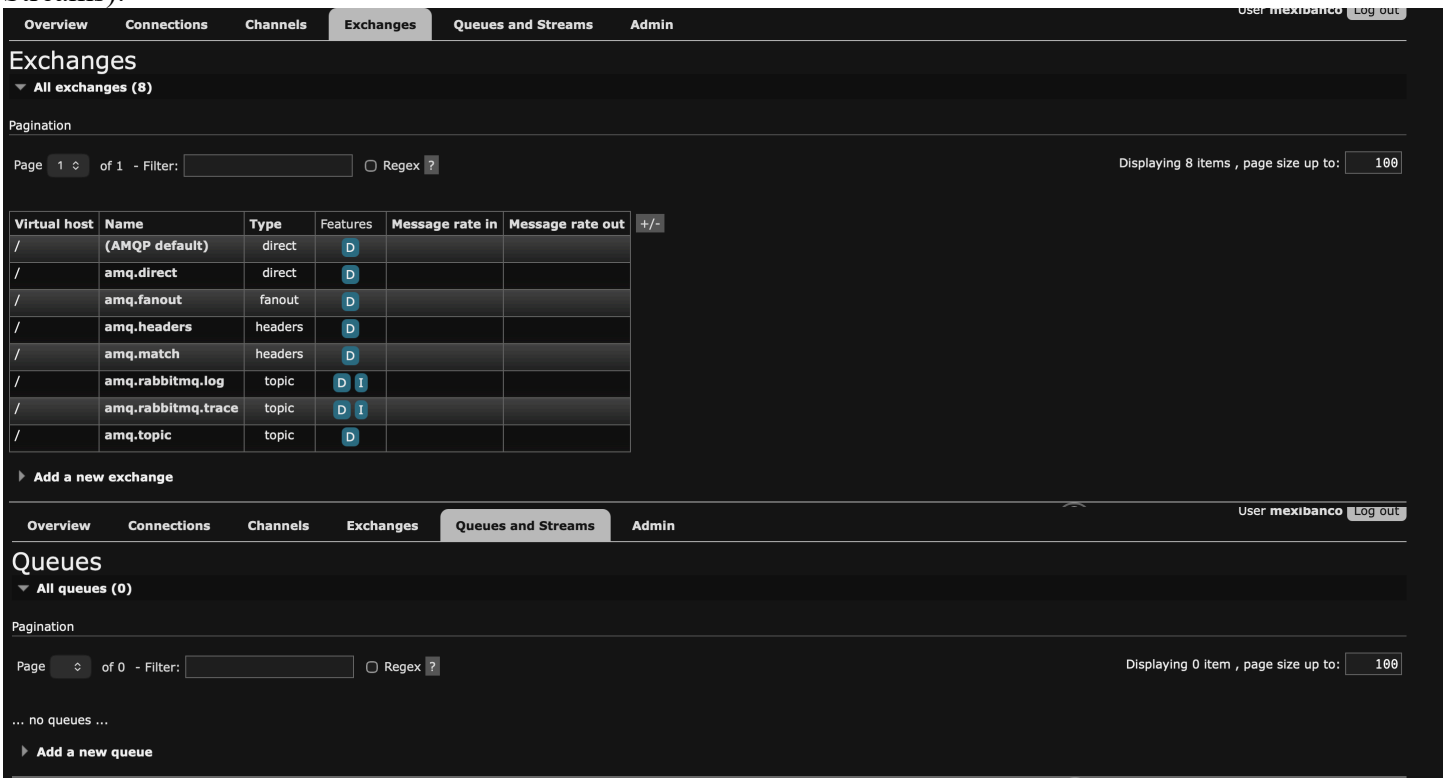
BITACORA – LAB07

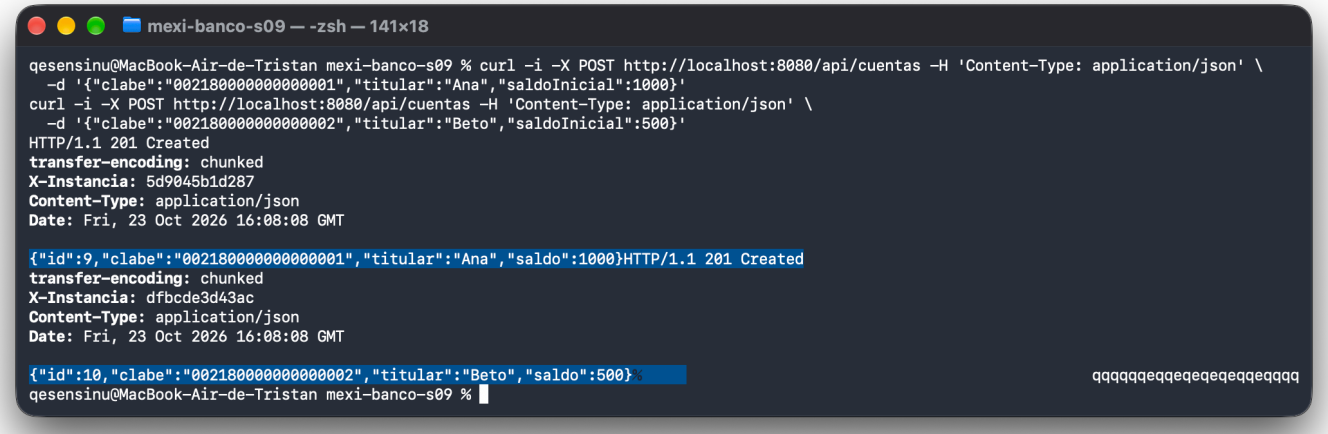
Alumno: Sanchez Mayen Tristan Qesen

⇒ PARTE 0. EL PROBLEMA

Para poder tener una referencia de lo que haremos hoy en el laboratorio es importante tener una evidencia de como empezo el sistema. A continuacion se muestran las capturas de RabbitMQ (Exchanges y Queues and Streams):

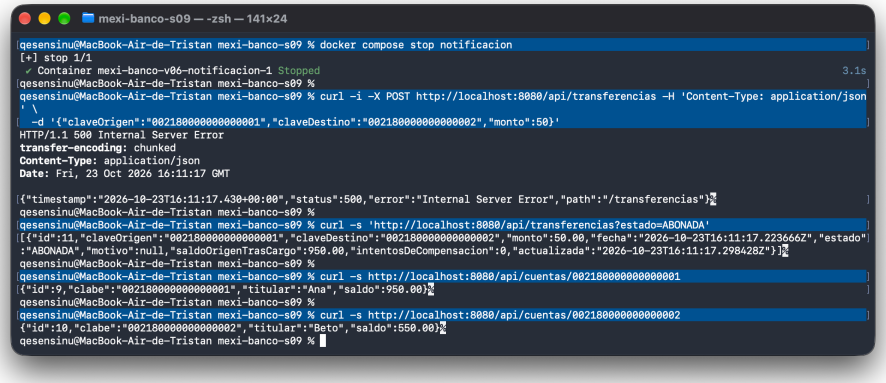


Ahora crearemos dos cuentas: Ana con $1000 y Beto con $500:



Para verificar que se crearon correctamente debemos ver el mensaje de “HTTP/1.1 **201 Created**” para ambas cuentas.

Ahora vamos a evidenciar el problema que se tiene. Para ello vamos a apagar notificacion y vamos a transferir



Podemos ver que al realizar la transferencia se dio el error 500 (Service Instance cannot be null).
Ademas al buscar la transferencia por el estado de “ABONADA”, esta aparece realizada
Al consultar los saldos de las cuentas verificamos que si se realizaron cambios en los saldos.

¿Por qué envia un error si la transferencia si se realizo?
La respuesta es porque en el flujo, Saga se queda esperando la respuesta de notificacion, pero como este servicio esta apagado no enviara dicha respuesta. Por eso es que se reporta un error aunque si se haya hecho la transferencia.

⇒ PARTE 1. PUBLICAR

Realizaremos las modificaciones necesarias para levantar mensaherua y ya no usar notificacion, para ello se editaron:

- `transferencia/src/main/java/mx/mexibanco/transferencia/Transferencia.java`
  Se elimino la dependencia a notificacion, para asegurar que la saga marque y guarde el estado COMPLETADA en la base de datos de PostgreSQL antes de emitir el evento, mientras que el método de avisar() captura cualquier fallo del broker en un bloque try-catch para evitar que un problema en la mensajería tire una transferencia cuyas operaciones financieras ya se completaron con éxito.

- `transferencia/src/main/resources/application.yml`
  Se definio un timeout, un publisher confirm para que Rabbit este obligado a responder y el template por si un evento no se puede enrutar a ninguna cola, debe devolverlo al publicador en lugar de descartarlo en silencio

Y se crearon:

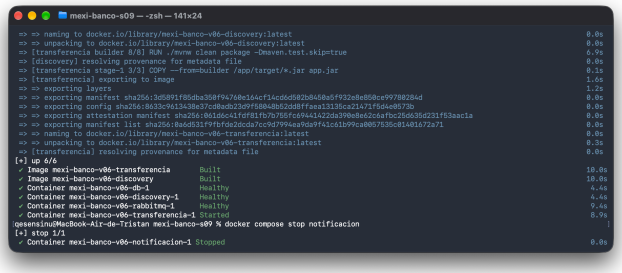
- `transferencia/src/main/java/mx/mexibanco/mensajeria/EventoTransferencia.java:`
  Lo que se hizo fue definir un record inmutable que actúa como contrato decoupling entre servicios, de esta forma se asigna un eventoId único desde su creación para garantizar la idempotencia.

- `mensajeria/ConfiguracionMensajeria.java`
  configura el convertidor JSON para serializar correctamente los eventos

- `mensajeria/EventoNoPublicadoException.java`
  Este sirve como excepción que funciona para representar explícitamente cualquier falla al intentar enviar o confirmar un evento en RabbitMQ

- `mensajeria/PublicadorDeEventos.java`
  Aquí se implemento la bandera mandatory, enviando el mensaje en modo persistente y esperando la confirmación explícita del broker antes de que se considere como completada una transferencia.

Ahora con los cambios hechos, vamos a reconstruir transferencia :



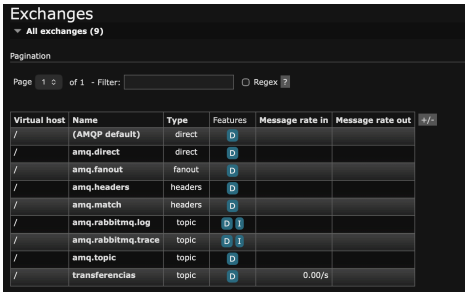
Ya que reconstruimos transferencia, ahora haremos una transferencia con notificacion apagada y lo que esperamos es un **201** lo que significa que la transferencia fue completada:



Y para saber que paso con el evento realizamos:



Dentro de RabbitMQ:



Activaste los publisher confirms. ¿Qué te garantiza la confirmación? ¿Y qué no te garantiza? Piensa en dos casos: que notificación esté apagado, y que el proceso de transferencia muera justo entre guardar(t) y avisar(...).

La confirmación nos garantiza que el mensaje salió del servicio, viajó por la red y fue recibido por el broker RabbitMQ. Saber que el broker devolvió un *ack* confirma que el evento está resguardado en la mensajería.
Lo que no nos garantiza es que el evento vaya a ser leído o procesado, tampoco garantiza atomicidad porque no une en una sola transacción la escritura en la base de datos con la publicación del mensaje en RabbitMQ.

⇒ PARTE 2. DESACOPLE EN TIEMPO

Ahora pasaremos las tres cosas que necesita notificacion (cola, consumidor y confirmar el mensaje despues de guardarlo) para que funcione con la mensajeria.
Realizaremos las modificaciones necesarias para que el servicio de notificación cree su cola, escuche los eventos de RabbitMQ y guarde los avisos, para ello se editaron:

- `notificacion/src/main/resorce/application.yml`
  Se cambió la confirmación de mensajes a modo manual para que la aplicación decida cuándo borrar el mensaje de la cola tras guardarlo en la base de datos, y se limitó la cantidad de mensajes recibidos a la vez.

- `notificacion/src/main/java/mx/mexibanco/notificacion/NotificacionService.java`

notificacion/src/main/java/mx/mexibanco/notificacion/NotificacionService.java
  Se agregaron los métodos necesarios para generar y guardar en la base de datos los avisos de envío, recepción o devolución dependiendo del tipo de evento recibido.

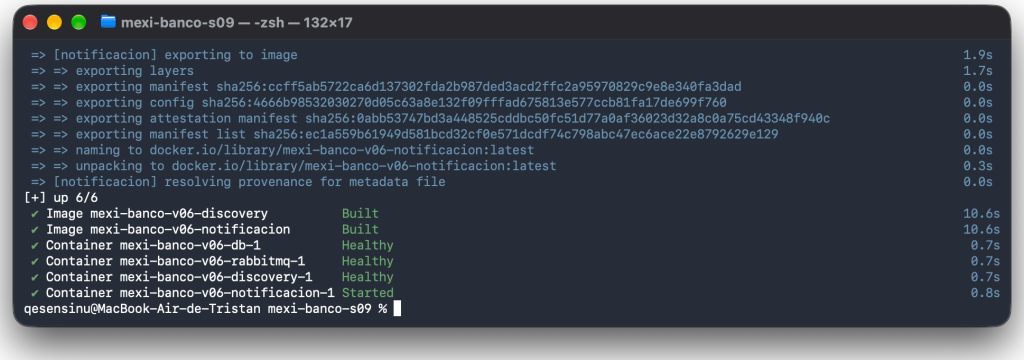
Y se crearon:

- `notificacion/src/main/java/mx/mexibanco/notificacion/mensajeria/ConfiguracionMensajeria.java`
  Se declaró la cola propia del servicio de notificación y se enlazó con el exchange mediante un patrón para a segurar que los mensajes esperen guardados en el broker hasta que el servicio pueda leerlos.

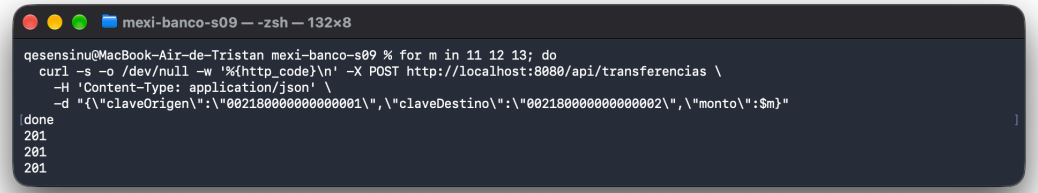
- `notificacion/src/main/java/mx/mexibanco/notificacion/EventoTransferencia.java`
  Se creó la copia local del contrato del evento dentro del servicio de notificación para poder interpretar los datos transmitidos en el formato JSON.

- `notificacion/src/main/java/mx/mexibanco/notificacion/mensajeria/ConsumidorDeTransferencias.java`
  Se implementó el listener que lee los mensajes de la cola, llama al servicio para registrar los avisos en la base de datos y envía la confirmación (ACK) manual a RabbitMQ únicamente después de guardar.

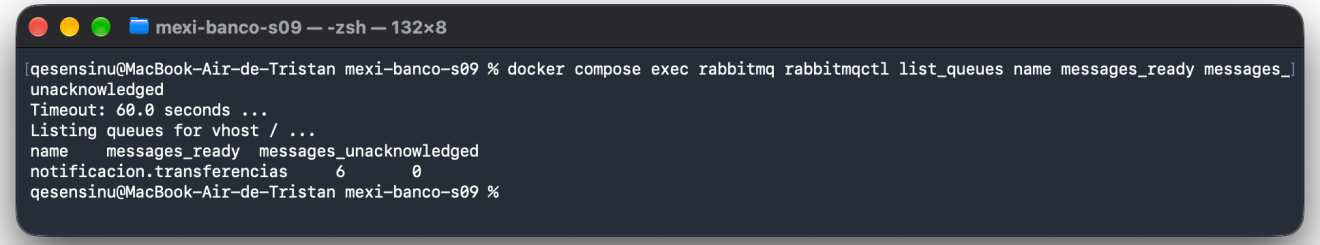
Ahora con los cambios hechos, vamos a reconstruir notificacion :



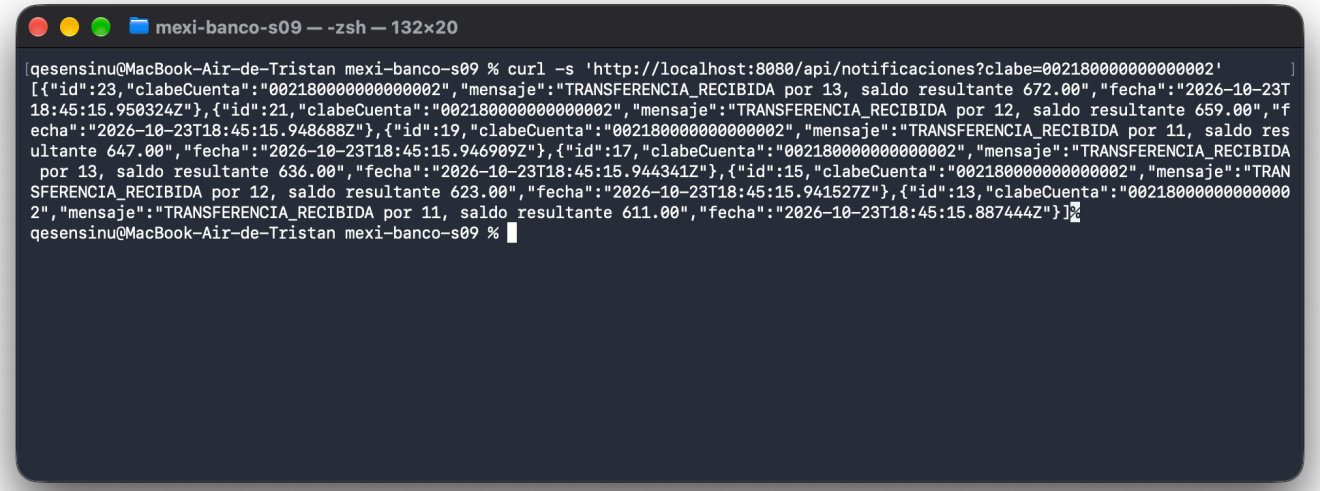
Una vez reconstruida, vamos a apagar notificacion y realizaremos 3 transferencias:



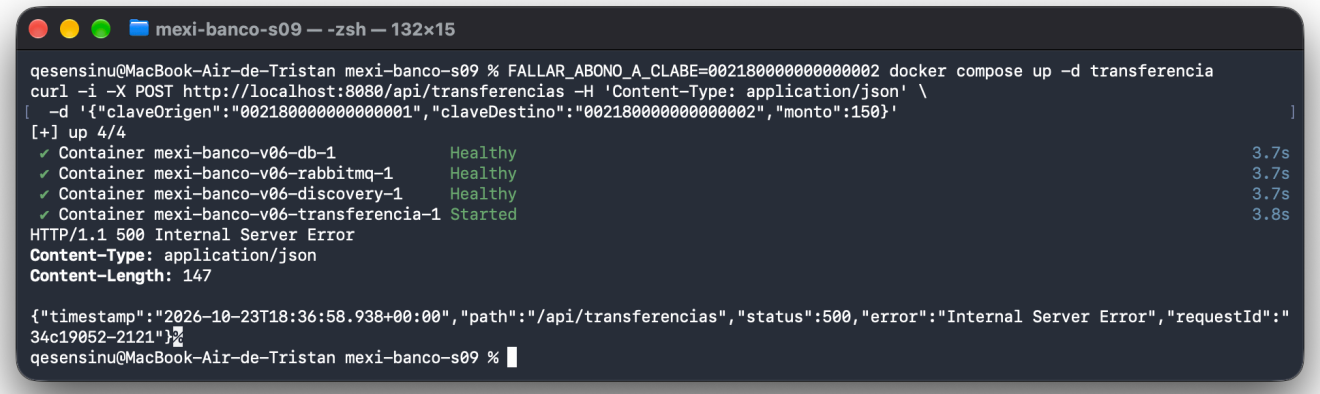
Ahora visualizaremos la cola:



Levantaremos de nuevo notificacion y veremos la bandeja de Beto:



Si ahora hacemos un caso de devolucion en caso de fallos, tambien se realiza:



¿Por qué el ack va después de guardar y no antes? Imagina que notificación muere justo entre las dos líneas, con cada orden. ¿Qué pasa con el aviso en cada caso: se pierde o se repite?

Porque el orden del ACK determina si privilegiamos no perder mensajes o evitar procesar duplicados a costa de perder avisos. Si se envia antes de guardar, el aviso se pierde de forma permanente, ya que nadie lo resguardó y RabbitMQ cree que ya fue procesado con éxito. Si se envia despues de guardar, el proceso se repetira.

⇒ PARTE 3. DUPLICADO

Ahora vamos a hacer que un evento envie un mensaje dos veces, tal como esta configurado actualmente el sistema (si recibe un evento dos veces, guardara dos juegos de avisos)
Para esto lo haremos matando el consumidor:

- Primero modificaremos `notificacion/src/main/resorce/application.yml` añadiendo una variable de pausa en la seccion de mexibanco

```
36  mexibanco:
37    pausa-antes-del-ack-ms: ${PAUSA_ANTES_DEL_ACK_MS:0}
38
```

- Luego en `docker-compose.yml` vamos a añadir un enviroment para notificacion:

```
212   notificacion:
213     <<: *servicio
214     build: ./notificacion
215     depends_on: *depende-del-broker
216     environment:
217       <<: [*entorno-base, *broker]
218       DB_NAME: notificacion
219       PAUSA_ANTES_DEL_ACK_MS: ${PAUSA_ANTES_DEL_ACK_MS:-0}
```

Realizaremos la comprobacion, primero reconstruyendo notificacion, despues realizando las transferencias para provocar los mensajes duplicados, despues mataremos notificacion, volveremos a levantar notificacion para verificar las notificaciones de Beto, tal como se muestra en la siguiente captura:

```
mexi-banco-s09 — -zsh — 132×10

qesensinu@MacBook-Air-de-Tristan mexi-banco-s09 % PAUSA_ANTES_DEL_ACK_MS=15000 docker compose up --build -d notificacion

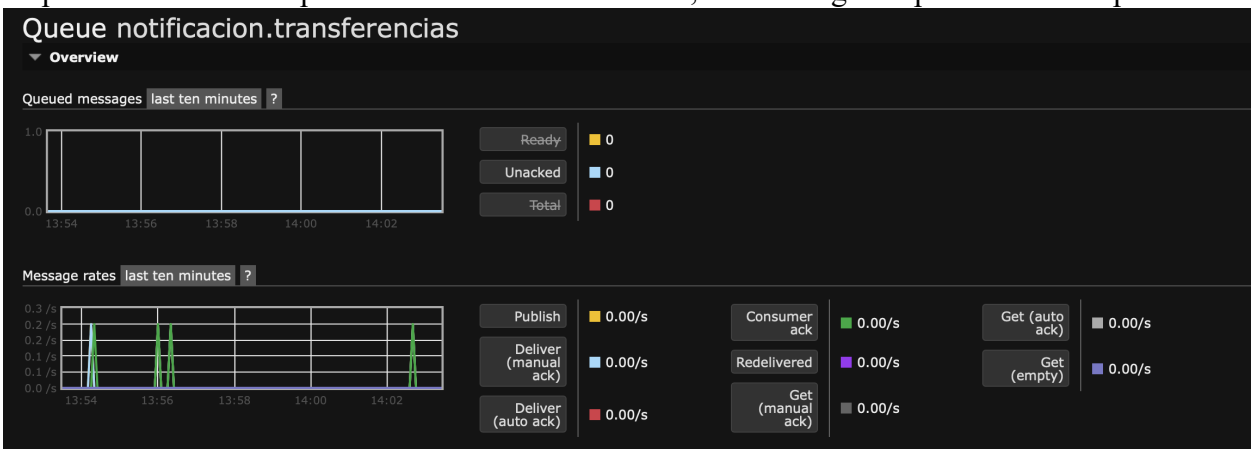
curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":80}'

docker compose kill notificacion

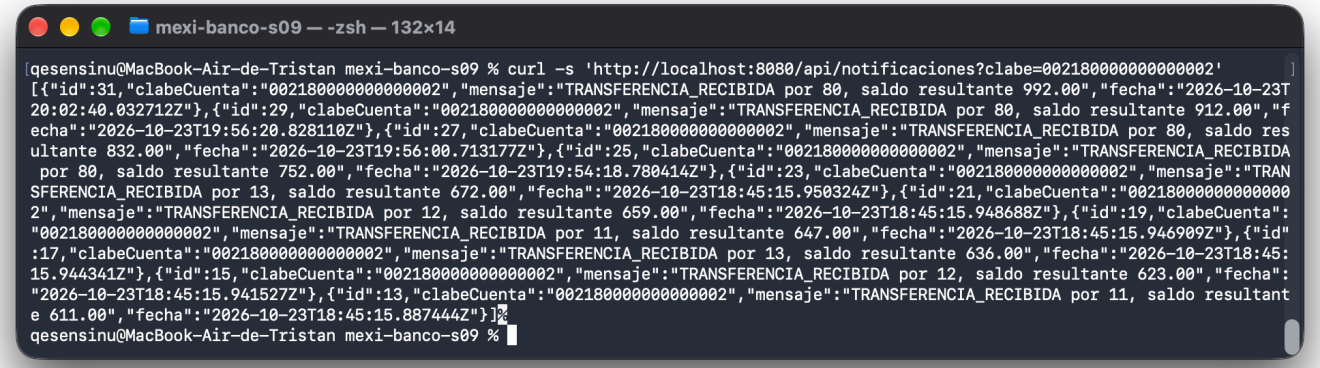
docker compose up -d notificacion

curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
```

Realice la prueba varias veces pero no me salto el 1 Unacked, sin embargo si aparece una recepcion de datos:

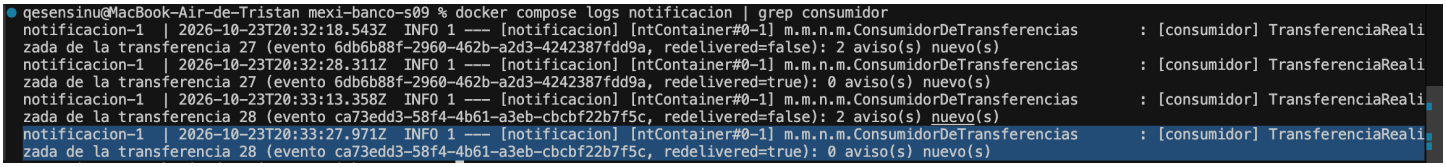


Las notificaciones de Beto son:



Para solucionar los mensajes repetidos tuve que crear otras entidades: EventoProcesado.java (Donde se realizara un registro de los procesos para tener una herramienta que nos ayude a garantizar la idempotencia), EventoProcesadoRepository.java (Es el que marcara un evento como repetido o como nuevo si es que no se ha repetido) y modificar NotificacionService.java (Para que se conecte con EventoProcesadoRepository.java), ConsumidorDeTransferencias.java (Para los eventos de reentrega) NotificacionServiceTest.java (Para verificar los eventos nuevos que se guardaron).

Una vez realizadas las modificaciones, realizaremos el mismo proceso de reconstruir notificacion, despues realizando las transferencias para provocar los mensajes duplicados, despues reiniciaremos notificacion y verificaremos los logs de las transferencias a Beto:



En la captura podemos ver que la notificacion aparece con un redelivered=true, lo que quiere decir que esta notificacion ha sido reenviada y que por lo tanto no debe duplicarse, solo mostrar una.

⇒ PARTE 4. LA DLQ

Ahora realizaremos un mensaje envenenado, lo que significa que es un mensaje que nunca se va a poder procesar y solo detendra el sistema. Para solucionar esto se debe enviar a una DLQ (Dead Letter Queue). Primero crearemos el mensaje directamente desde RabbitMQ → Exchanges → Publish message y en Routing key pondremos transferencia.realizada, luego en Payload {“hola”:

Si revisamos los logs, podemos ver que lo que nos dio el envio de este mensaje fue lo siguiente:

```
mexi-banco-s09 — docker-compose ◂ docker compose logs -f notificacion — 189×55

notificacion-1  |
notificacion-1  | org.springframework.amqp.rabbit.support.ListenerExecutionFailedException: Listener method 'public void mx.mexibanco.notificacion.mensajeria.ConsumidorDeTransferencias.recibir(org.springframework.amqp.core.Message,com.rabbitmq.client.Channel) throws java.io.IOException' threw exception
notificacion-1  |       at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.invokeHandler(MessagingMessageListenerAdapter.java:287) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.invokeHandlerAndProcessResult(MessagingMessageListenerAdapter.java:225) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.onMessage(MessagingMessageListenerAdapter.java:149) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.doInvokeListener(AbstractMessageListenerContainer.java:1682) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.actualInvokeListener(AbstractMessageListenerContainer.java:1604) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.invokeListener(AbstractMessageListenerContainer.java:1592) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.doExecuteListener(AbstractMessageListenerContainer.java:1583) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.executeListenerAndHandleException(AbstractMessageListenerContainer.java:1528) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.lambda$executeListener$8(AbstractMessageListenerContainer.java:1506) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at io.micrometer.observation.Observation.observe(Observation.java:499) ~[micrometer-observation-1.13.4.jar!/:1.13.4]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.AbstractMessageListenerContainer.executeListener(AbstractMessageListenerContainer.java:1506) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer.doReceiveAndExecute(SimpleMessageListenerContainer.java:1082) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer.receiveAndExecute(SimpleMessageListenerContainer.java:1018) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer$AsyncMessageProcessingConsumer.mainLoop(SimpleMessageListenerContainer.java:1421) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.SimpleMessageListenerContainer$AsyncMessageProcessingConsumer.run(SimpleMessageListenerContainer.java:1322) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at java.base/java.lang.Thread.run(Unknown Source) ~[na:na]
notificacion-1  | Caused by: com.fasterxml.jackson.core.io.JsonEOFException: Unexpected end-of-input within/between Object entries
notificacion-1  |  at [Source: REDACTED (`StreamReadFeature.INCLUDE_SOURCE_IN_LOCATION` disabled); line: 2, column: 1]
notificacion-1  |       at com.fasterxml.jackson.core.base.ParserMinimalBase._reportInvalidEOF(ParserMinimalBase.java:585) ~[jackson-core-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.core.json.UTF8StreamJsonParser._skipColon2(UTF8StreamJsonParser.java:3226) ~[jackson-core-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.core.json.UTF8StreamJsonParser._skipColon(UTF8StreamJsonParser.java:3141) ~[jackson-core-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.core.json.UTF8StreamJsonParser.nextToken(UTF8StreamJsonParser.java:761) ~[jackson-core-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.databind.deser.BeanDeserializer.deserialize(BeanDeserializer.java:181) ~[jackson-databind-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.databind.deser.DefaultDeserializationContext.readRootValue(DefaultDeserializationContext.java:342) ~[jackson-databind-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.databind.ObjectMapper._readMapAndClose(ObjectMapper.java:4905) ~[jackson-databind-2.17.2.jar!/:2.17.2]
notificacion-1  |       at com.fasterxml.jackson.databind.ObjectMapper.readValue(ObjectMapper.java:3909) ~[jackson-databind-2.17.2.jar!/:2.17.2]
notificacion-1  |       at mx.mexibanco.notificacion.mensajeria.ConsumidorDeTransferencias.recibir(ConsumidorDeTransferencias.java:39) ~[!/:0.9.0]
notificacion-1  |       at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(Unknown Source) ~[na:na]
notificacion-1  |       at java.base/java.lang.reflect.Method.invoke(Unknown Source) ~[na:na]
notificacion-1  |       at org.springframework.messaging.handler.invocation.InvocableHandlerMethod.doInvoke(InvocableHandlerMethod.java:169) ~[spring-messaging-6.1.13.jar!/:6.1.13]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.adapter.KotlinAwareInvocableHandlerMethod.doInvoke(KotlinAwareInvocableHandlerMethod.java:45) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.messaging.handler.invocation.InvocableHandlerMethod.invoke(InvocableHandlerMethod.java:119) ~[spring-messaging-6.1.13.jar!/:6.1.13]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.adapter.HandlerAdapter.invoke(HandlerAdapter.java:75) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       at org.springframework.amqp.rabbit.listener.adapter.MessagingMessageListenerAdapter.invokeHandler(MessagingMessageListenerAdapter.java:278) ~[spring-rabbit-3.1.7.jar!/:3.1.7]
notificacion-1  |       ... 15 common frames omitted
notificacion-1  |
```

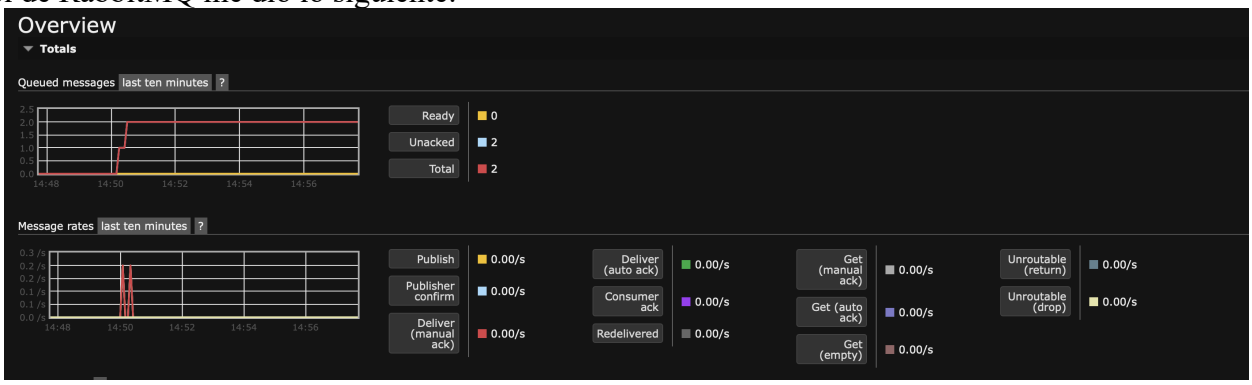
¿El mensaje rebota, se queda trabado en Unacked o tira el consumidor? ¿Una transferencia normal después de él sigue generando avisos? Descríbelo con los logs y una captura del panel.

El mensaje provoca que el proceso Java no se caiga ni se apague, el contenedor notificacion sigue corriendo. Al no haber un bloque try-catch que maneje el error de deserialización de Jackson, Spring AMQP atrapa la excepción inédita y, por omisión, reencola el mensaje.

El consumidor intenta leer el mensaje malo, luego falla al parsear el JSON, lanza la excepción, RabbitMQ vuelve a entregar el mensaje pero este vuelve a fallar. El mensaje conmuta constantemente entre Ready y Unacked en un bucle infinito de reintentos.

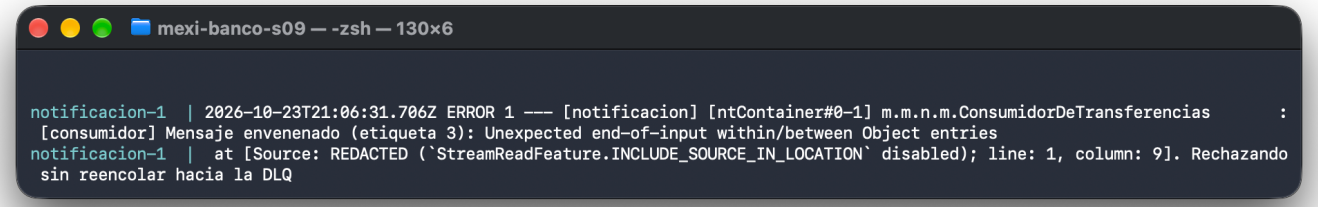
Una transferencia despues de este mensaje si sigue generando avisos pero con retraso/bloqueo: Como la cola procesa los mensajes en orden secuencial (FIFO), el mensaje envenenado que está al frente se queda atorado intentando reprocesarse infinitamente. Una transferencia normal que llegue después quedará encolada detrás del mensaje corrupto hasta que este sea consumido, descartado o derivado a una DLQ.

El panel de RabbitMQ me dio lo siguiente:

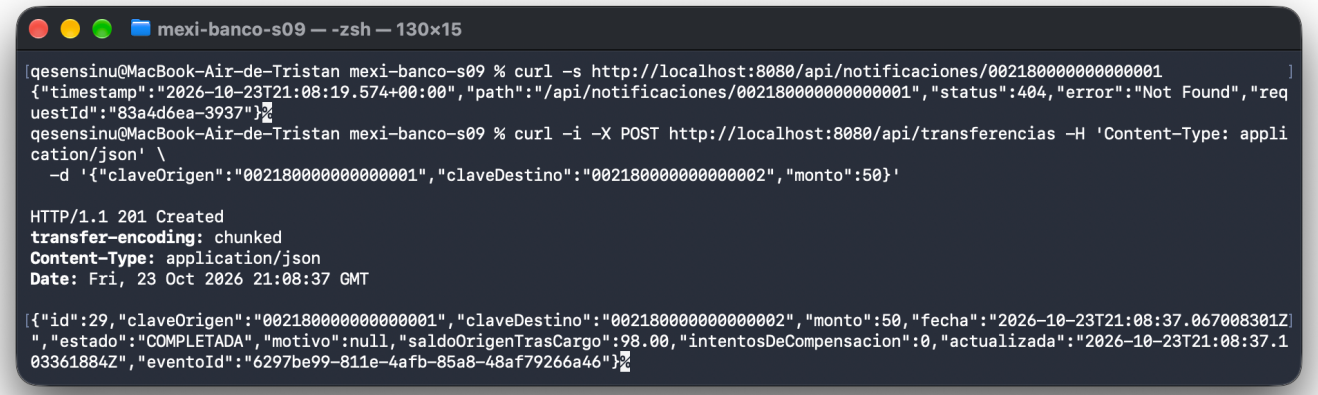


Para implementar la DLQ debemos de modificar ConfiguracionMensajeria.java (Se deben configurar los de la cola principal para despues redirigir rechazos), ConsumidorDeTransferencias.java (Se debe colocar la deserializacion en un try-catch)

Para poner a prueba las modificaciones tendremos que reconstruir notificacion y luego enviar el mensaje envenenado de nuevo mientras revisamos el log. Este es el resultado:



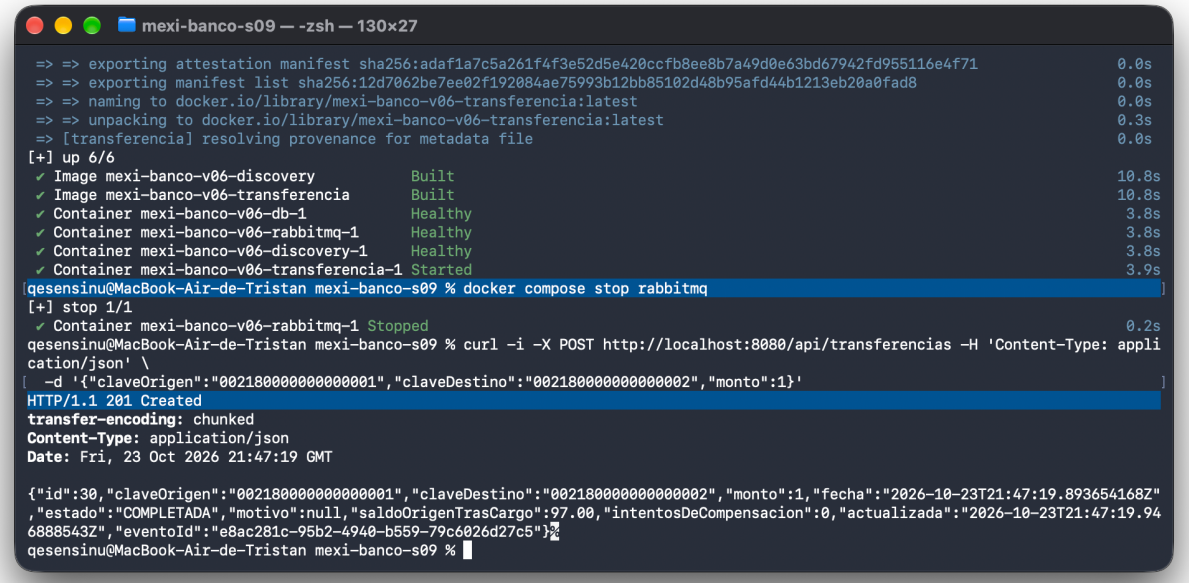
Podemos ver que los mensajes envenenados si son enviador a la DLQ, pero aun falta ver que las transferencias sigan enviando avisos con normalidad:



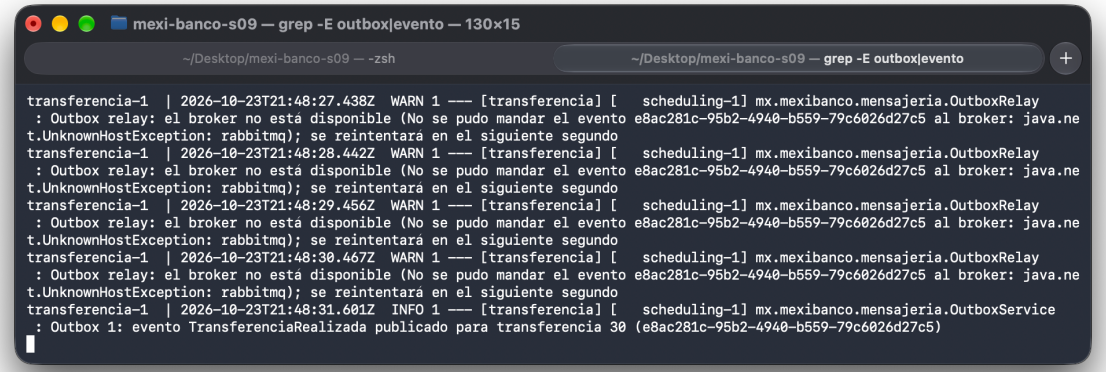
⇒ PARTE 5. OUTBOX

Ahora vamos a para el servicio de transferencia ha quedado completamente implementado, resolviendo el problema de la doble escritura y garantizando que ningún evento se pierda aun si RabbitMQ está completamente apagado cuando la saga llega a COMPLETADA.
Para esto se crearon EventoOutbox.java (Al ser una tabla de PostgreSQL dentro del mismo servicio de transferencia, permite que la creación del evento se guarde en la misma transacción ACID que el cambio de estado de la transferencia a COMPLETADA o COMPENSADA.), EventoOutboxRepository.java (Es un querry que se encarga de consultar la concurrencia segura con FOR UPDATE SKIP LOCKED), OutboxService.java, OutboxRelay.java y una modificacion en TransferenciaService.java (estos ultimos sirven para que haya Atomicidad con TransactionTemplate y el desacoplamiento del brocker RabbitMQ, si el broker está caído, la transferencia responde 201 Created normalmente y el evento queda a salvo en la tabla outbox).

Una vez realizadas las modificaciones anteriores, debemos reconstruir transferencia, detener RabbitMQ y realizar transferencias con Rabbit apagado. Lo que esperamos es que las transferencias se marquen como completas con el codigo 201:



Ahora activaremos RabbitMQ y revisaremos el log que se realiza:



Lo que sucedió es que se enviaban mensajes a RabbitMQ hasta que respondiera, una vez que se levanto se pudo confirmar la transferencia, sin embargo no se tuvo que mantener al usuario esperando por la transferencia ya que esta se registraba en la base de datos del Outbox.

⇒ PARTE 6. PREGUNTAS DE CIERRE

1. **Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?**
   En esta versión con Outbox, el aviso se guarda en la tabla outbox en la misma transacción local que el cambio de estado de la transferencia. Cuando RabbitMQ regresa, el relay programado lo envía automáticamente sin perderlo.
   El publisher confirm por sí solo no lo arregla porque únicamente confirma la recepción o rechazo una vez que el mensaje llega al broker conectado, si el broker está caído o inaccesible, el intento de publicación arroja error de conexión o timeout. Si la publicación se hiciera en caliente al terminar la saga, la falla ocurriría tras haber confirmado la transferencia en la base (problema clásico de doble escritura sin transacción distribuida), perdiendo el aviso irremediablemente.

2. **¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?**
   ACK antes de guardar notifica al broker que elimine el mensaje antes de asegurar su persistencia. Si el consumidor se cae o la base de datos falla al guardar, el mensaje ya no existe en la cola y se pierde para siempre.
   ACK después de guardar garantiza que el mensaje solo se retira del broker una vez que la transacción en la base de datos hizo commit. Si el consumidor muere antes del ACK, RabbitMQ lo reentrega (redelivered=true). La tabla de idempotencia evita que esta reentrega genere duplicados.

3. **Elección, por qué y qué se sacrificó**
   Eleccion: Se eligio la comunicación asíncrona mediante broker y patrón Transactional Outbox.
   Porque: Porque el aviso al cliente es un efecto secundario que no debe detener ni abortar la transferencia del dinero si el sistema de notificaciones o el broker sufren una falta de disponibilidad.
   Sacrifico: Consistencia inmediata; la notificación ya no se refleja instantáneamente en el mismo milisegundo de la respuesta HTTP, sino tras una breve latencia de propagación (consistencia eventual).